

DSCD 614 - Reinforcement Learning

Programming Assignment 1 - Technical Report

Frozen Lake from First Principles Using Q-Learning

Name: GILBERT WOASIEWORM ADJEI

Student ID: 22425084

GitHub Repository:

<https://github.com/Woasieworm/frozen-lake-qlearning>

Report Link:

https://drive.google.com/drive/folders/1SWib5_7j_9tiA4nz3OI9xR0etyHsFm0?usp=sharing

1. Introduction

Reinforcement Learning (RL) is a branch of machine learning where an agent learns to make decisions by interacting with an environment and receiving reward signals. Unlike supervised learning, the agent is not given correct answers. It explores on its own, figures out which actions lead to good outcomes, and gradually builds a policy that maximises its total reward over time.

This report covers the design and implementation of a Q-Learning agent that solves the 8x8 Frozen Lake problem entirely from scratch in Python. No RL frameworks were used. The report describes the environment, the algorithm, the training setup, and the results, and also discusses some of the practical challenges that came up during development.

2. Environment Design

2.1 State and Action Representation

The environment is an 8x8 grid. The agent starts at the top-left corner (S) and must reach the goal at the bottom-right corner (G) without stepping into any hole (H). Each cell is encoded as a single integer using the formula below, giving 64 states in total.

```
state = row x 8 + col      (range: 0 to 63)
```

The agent has four possible actions at every step: 0 = Left, 1 = Down, 2 = Right, 3 = Up. If an action would push the agent off the grid, it stays in its current cell.

2.2 Reward Structure

Outcome	Reward	Purpose
Reached the Goal (G)	+1.0	The primary signal the agent is trying to maximise
Fell in a Hole (H)	-1.0	Discourages risky navigation
Moved on safe ice (F)	-0.01	Encourages reaching the goal quickly

Table 1: Reward structure used in the Frozen Lake environment.

The -0.01 step penalty was added to address the sparse-reward problem. Without it, every non-terminal step returns zero and the Q-table has nothing to learn from until the agent accidentally finds the goal, which happens only about 0.3% of the time on this map. The penalty gives every transition a meaningful value, which allows Q-values to start propagating from the goal backwards through the grid much earlier in training.

3. Q-Learning Algorithm

Q-Learning (Watkins & Dayan, 1992) is a model-free, off-policy algorithm that maintains a table of Q-values. Each entry $Q(s, a)$ estimates the total discounted reward the agent can expect by taking action a in state s and then acting optimally. After every step, the relevant Q-value is updated using the equation specified in the assignment:

```
 $Q(s, a) \leftarrow Q(s, a) + \alpha * [ r + \gamma * \max_{a'} Q(s', a') - Q(s, a) ]$ 
```

The term in brackets is the TD error, which measures how wrong the current estimate was. Alpha controls how quickly the agent updates its estimates, and gamma controls how much it values future rewards. When an episode ends, the future value is set to zero because there are no more steps.

Action selection uses epsilon-greedy exploration. With probability epsilon the agent picks a random action, and otherwise it picks the action with the highest Q-value. Epsilon is decayed after each episode so the agent gradually shifts from exploring to exploiting its learned knowledge.

4. Training Methodology

Parameter	Value	Reasoning
Learning rate (alpha)	0.3	Faster convergence than 0.1, remains stable
Discount factor (gamma)	0.99	Places high value on reaching the distant goal
Starting epsilon	1.0	Agent starts by exploring fully at random
Minimum epsilon	0.01	Keeps a small amount of exploration after convergence
Epsilon decay	0.9999079	Epsilon reaches minimum around episode 50,000
Episodes	50,000	Gives Q-values enough time to propagate fully
Max steps per episode	500	Long enough to reach the goal from any position

Table 2: Hyperparameters used for training the Q-Learning agent.

The most critical decision was the epsilon decay schedule. A common default of 0.995 brings epsilon below 0.01 by episode 1,000, which is far too fast for an 8x8 map. At that point the agent has barely seen the goal, so its Q-table is almost entirely zeros. When it switches to exploitation it just repeats the same bad moves. By keeping epsilon above 0.3 until around episode 15,000, the agent has enough time to discover the goal repeatedly and build up a useful Q-table before committing to a greedy policy.

The evaluation was always run in a separate FrozenLakeEnv instance, never in the training environment. This matters because passing the training environment into the evaluation function would corrupt its state when reset() is called. Keeping them separate ensures the training loop is never interrupted.

5. Experimental Results

5.1 Training Progress

A greedy evaluation was run every 5,000 episodes using 200 test episodes with epsilon set to zero. The agent reached 100% success from the very first checkpoint at episode 5,000 and held it for the rest of training.

Episode	Train Wins / 5,000	Greedy Success Rate	Epsilon
5,000	280	100.0%	0.6310
10,000	1,176	100.0%	0.3981
20,000	3,313	100.0%	0.1585
30,000	4,304	100.0%	0.0631
40,000	4,714	100.0%	0.0251
50,000	4,894	100.0%	0.0100

Table 3: Selected training checkpoints. Full results available in results/train_stats.json.

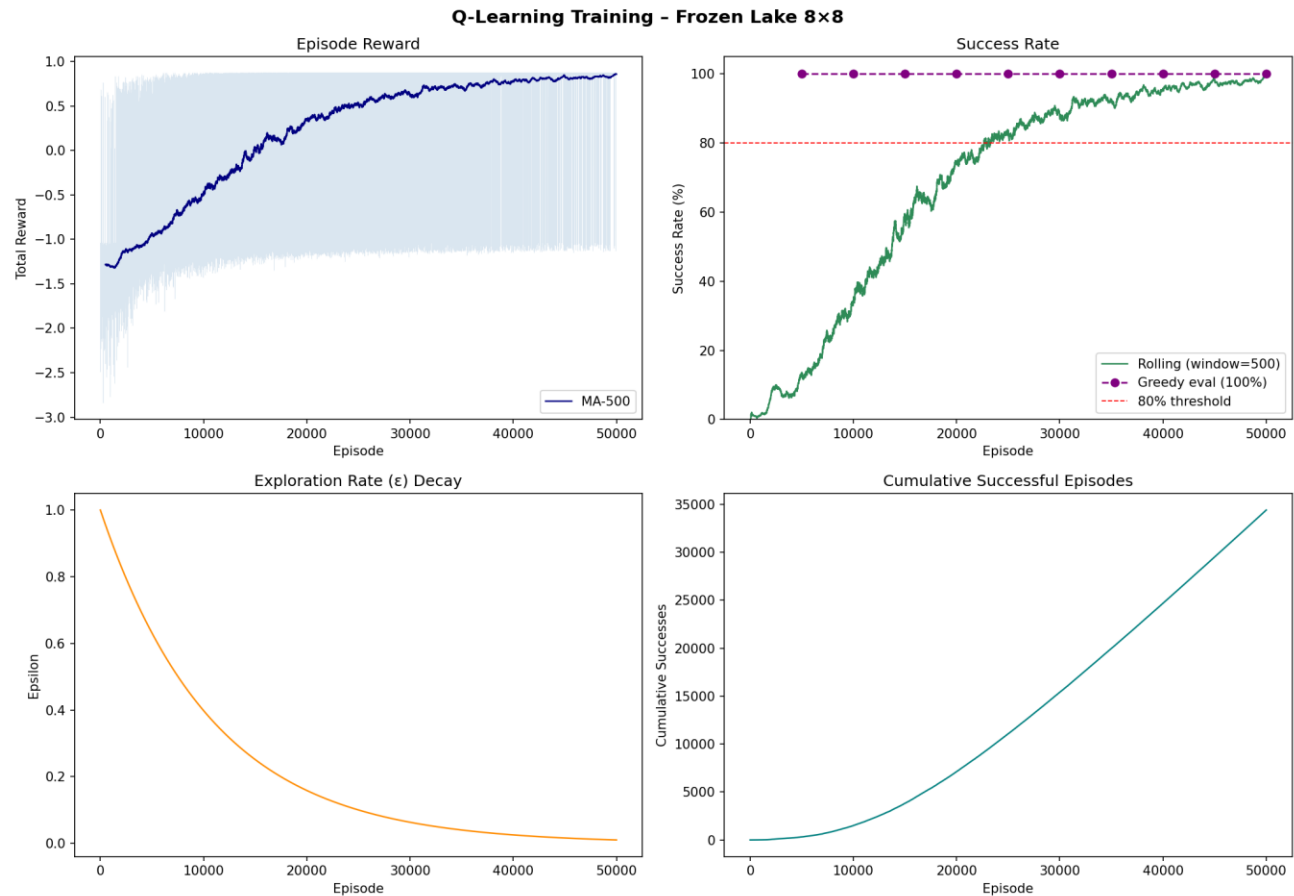


Figure 1: Training progress across 50,000 episodes (reward, success rate, epsilon decay, cumulative wins).

5.2 Final Evaluation

Metric	Result
Episodes run	500
Successful	500
Failed	0
Success rate	100.00%
Average reward	0.87

Table 4: Final greedy evaluation over 500 episodes.

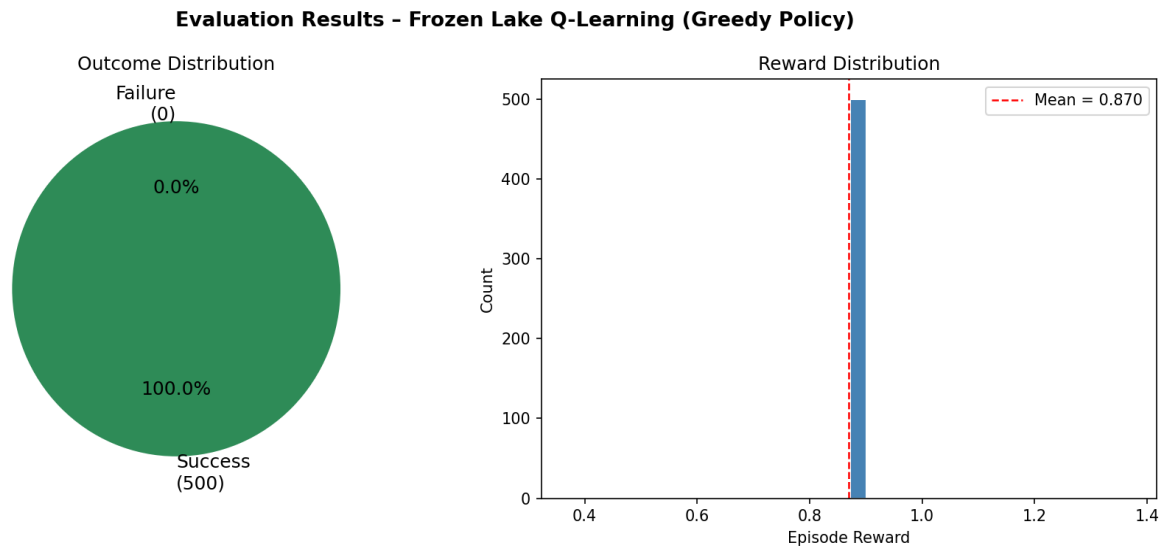


Figure 2: Outcome distribution and reward histogram from the 500-episode evaluation.

6. Learned Policy

The policy below was extracted by taking the highest Q-value action in every non-terminal state. H marks holes and G marks the goal.

```
down down down down down down down down
right right right right down down down down
right right up H down down down down
right right up H right right right down
up H H right right up H down
up H right up H up H down
up left down H right up right G
```

The agent moves right and downward through the top of the grid to reach column 7, then travels straight down along that column to the goal. This path avoids all ten holes and the agent follows it successfully in every evaluation episode.

6.1 Exploration Strategy Comparison (Bonus C)

Pure Epsilon-Greedy (epsilon fixed at 0.1) and Decaying Epsilon-Greedy (1.0 to 0.01) were both trained and evaluated. Both strategies reached 100% greedy success. Pure Epsilon-Greedy accumulated more training wins early on because it was exploiting 90% of the time from the start. Decaying Epsilon-Greedy started lower but ended slightly higher, and produces a fully deterministic policy at the end rather than one that still makes random decisions 10% of the time.

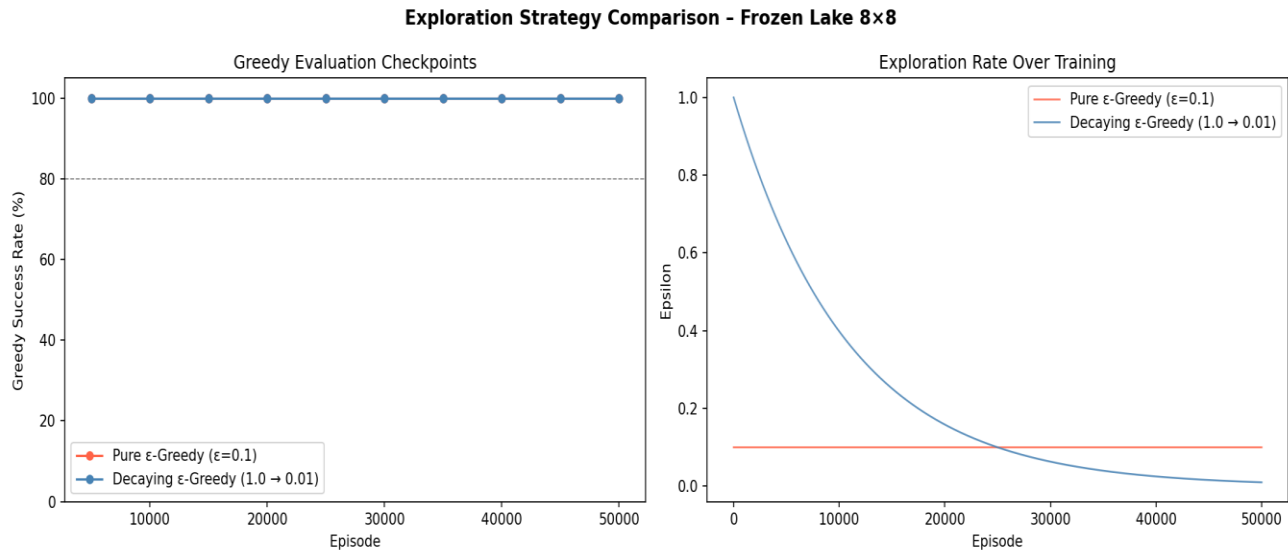


Figure 3: Greedy success rate and epsilon schedule for both exploration strategies.

7. Challenges Encountered

Sparse rewards. With zero reward for non-terminal steps, the Q-table stayed flat because the agent was not reaching the goal often enough to push any value through the table. Adding the -0.01 step penalty gave every transition a non-zero value and allowed learning to begin immediately.

Epsilon decay speed. A decay of 0.995 caused epsilon to collapse below 0.01 before the Q-table had any meaningful values. The agent locked into a bad greedy policy and never recovered. Slowing the decay to 0.9999079 kept exploration alive long enough for the Q-table to become reliable.

Shared environment state. Early versions passed the training environment into the evaluation function, which called `reset()` and corrupted the training state. The fix was to always create a fresh environment inside each evaluation call.

8. Conclusion

A complete Q-Learning solution for the 8x8 Frozen Lake problem was built from first principles in Python. The agent reached a 100% success rate across 500 greedy evaluation episodes after 50,000 training episodes. The key factors were the step penalty (which solved the sparse-reward problem), the slow epsilon decay (which kept exploration alive long enough), and a separate evaluation environment (which kept the training loop clean). All three bonus options were also completed: stochastic transitions (Option A), training visualisations (Option B), and an exploration strategy comparison (Option C).

References

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3-4), 279-292.